```
{
    public IActionResult ManageAccount()          ❷
    {
        return Ok();
    }
    [AllowAnonymous]                              ❸
    public IActionResult Login()                  ❹
    {
        return Ok();
    }
}
```

❶ Applied at the controller scope, so the user must be authenticated for all actions on the controller

❷ Only authenticated users may execute ManageAccount.

❸ [AllowAnonymous] overrides [Authorize] to allow unauthenticated users.

❹ Login can be executed by anonymous users.

WARNING If you apply the `[Authorize]` attribute globally, be sure to add the `[AllowAnonymous]` attribute to your login actions, error actions, password reset actions, and any other actions that you need unauthenticated users to execute. If you're using the default Identity UI described in chapter 23, this is already configured for you.

If an unauthenticated user attempts to execute an action protected by the `[Authorize]` attribute, traditional web apps redirect them to the login page. But what about APIs that don't have a user interface? And what about more complex scenarios, where a user is logged in but doesn't have the necessary claims to execute an action? In section 24.2.2 we'll look at how the ASP.NET Core authentication services handle all this for you.

24.2.2 Handling unauthorized requests

In the previous section you saw how to apply the `[Authorize]` attribute to an action to ensure that only authenticated users can execute it. In section 24.3 we'll look at more complex examples that require you to also